

Relatório de Auditoria de Segurança

Horium

Data: 01/09/2026

Escopo: Código-fonte completo (frontend + schema SQL)

Stack: React 19 + Vite + Supabase (PostgreSQL + Auth + Storage)

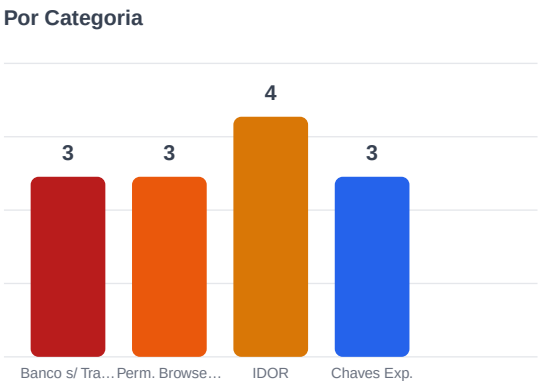
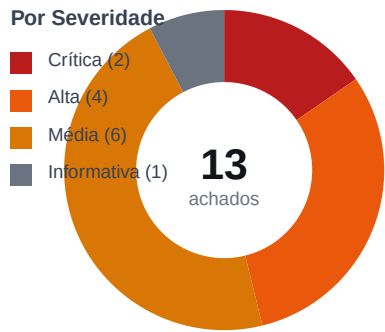
Backend: Supabase RLS + RPCs (SECURITY DEFINER)

Frontend: React SPA (TypeScript) com TailwindCSS

Deploy: Sem Docker/CI/Helm/Terraform detectados

Nota Metodológica: Cada categoria de auditoria foi mapeada para a stack Supabase — "Banco sem Tranca" = RLS ausente/furado; "Permissão no Navegador" = gate frontend sem validação na RPC; "IDOR" = queries sem filtro user_id; "Chaves Expostas" = segredos no código/git; "XSS" = padrões inseguros de renderização no React.

Resumo Executivo



Pontos Fortes

- [S01] RLS ativo em TODAS as tabelas — schedules, licenses, notifications, tickets têm Row Level Security habilitado com políticas corretas que filtram por `auth.uid() = user_id`.
Evidência: `schema.sql:27` (schedules), 84 (licenses), 121 (notifications), 148 (tickets)
- [S02] RPCs administrativas com SECURITY DEFINER e validação `is_admin()` — `get_admin_licenses`, `approve_license_rpc`, `delete_license_rpc`, `get_pending_licenses_count_rpc` todas verificam `is_admin()` antes de executar.
Evidência: `schema.sql:166-283`
- [S03] Trigger de proteção contra manipulação de `is_licensed` — `protect_schedule_license_status()` impede que não-admins alterem o campo `is_licensed` diretamente.
Evidência: `schema.sql:47-65`
- [S04] Storage buckets PRIVADOS com RLS — `receipts` e `tickets-attachments` são privados (`public=false`) com políticas de isolamento por `user_id` via `storage.foldername()`.
Evidência: `schema.sql:286-334`
- [S05] Autenticação via Supabase Auth — fluxo de sign-up com verificação por OTP, reset de senha via link, persistência de sessão com auto-refresh.
Evidência: `services/authService.ts` (todo o arquivo)
- [S06] Sem XSS: React impede innerHTML por default — o codebase não utiliza `dangerouslySetInnerHTML`, `innerHTML`, `v-html`, `eval()` ou `new Function()` em NENHUM arquivo.
Evidência: Busca grep em todo o projeto retornou 0 resultados para todos os padrões XSS.
- [S07] `.env.local` no `.gitignore` — o arquivo de variáveis de ambiente local está excluído do rastreamento git via padrão `*.local`.
Evidência: `.gitignore:13`
- [S08] Validação de insert em licenses — RLS permite INSERT apenas com `payment_status` IN ('Aguardando', 'under_review') ou NULL, impedindo que usuários criem licenças já aprovadas.
Evidência: `schema.sql:101-107`
- [S09] Upload de arquivos com sanitização de extensão — extensões são limpas com regex (remove caracteres não alfanuméricos) e limitadas a 5 caracteres.
Evidência: `services/ticketService.ts:21-22`, `pages/PlansPage.tsx:132-133`
- [S10] Headers de segurança no `index.html` — X-Content-Type-Options: nosniff e referrer policy strict-origin-when-cross-origin.
Evidência: `index.html:7-8`

Pontos Fracos (Riscos Centrais)

1. Chave de API Google Gemini comprometida no histórico git — exploração imediata possível.

2. Padrão de fallback client-side em operações admin — dependência frágil no RLS.
3. E-mails de admin hardcoded em código-fonte público — exposição de identidade.
4. URLs públicas geradas para buckets privados — funcionalidade de visualização quebrada.
5. Queries sem filtro explícito de user_id — dependência total no RLS para isolamento.

Achados Detalhados

1. Banco sem Tranca (RLS)

ALTA

F01 — services/adminLicenseService.ts:144-158

Contagem de licenças pendentes sem filtro de admin — qualquer usuário autenticado pode consultar.

Explorabilidade: O RLS da tabela limita o impacto: usuários comuns verão apenas suas próprias licenças. Porém, a RPC segura é ignorada, criando dependência frágil no RLS.

```
async getPendingLicensesCount(): Promise<number> {
  const { count, error } = await supabase
    .from('licenses')
    .select('*', { count: 'exact', head: true })
```

MÉDIA

F02 — services/ticketService.ts:36-40

URL pública gerada para bucket PRIVADO — getPublicUrl() gera URL que não funciona para buckets privados, mas o valor é salvo na tabela.

Explorabilidade: Impacto funcional mais que de segurança: imagens de tickets não serão visíveis. A URL salva não permite acesso real ao arquivo.

```
const { data } = supabase.storage
  .from('tickets-attachments')
  .getPublicUrl(filePath);
imageUrl = data.publicUrl;
```

MÉDIA

F03 — pages/PlansPage.tsx:146-149

Mesmo problema de getPublicUrl() em bucket privado para comprovantes de pagamento (receipts).

Explorabilidade: Idem ao F02. Risco funcional, não de exposição de dados.

```
const { data: publicUrlData } = supabase.storage
  .from('receipts')
  .getPublicUrl(filePath);
receiptUrl = publicUrlData?.publicUrl || null;
```

2. Permissão no Navegador

CRÍTICA

F04 — components/Layout.tsx + App.tsx:Layout:66,180 / App:438-439

O gate de acesso à área administrativa é EXCLUSIVAMENTE no frontend (lista hardcoded de e-mails). Qualquer usuário autenticado pode navegar diretamente para a view ADMIN manipulando o estado.

Explorabilidade: Impacto reduzido porque as RPCs do banco validam admin. Porém, é má prática: o frontend expõe a UI e pode revelar informações pela tentativa de chamada.

```
// Layout.tsx:66
setIsAdmin(ADMIN_EMAILS.includes(user.email || ''));
// Layout.tsx:180 - Renderiza botão Admin apenas se isAdmin
{isAdmin && (<button onClick={() => setView(AppView.ADMIN)}>...)}
```

ALTA

F05 — services/adminLicenseService.ts:56-101

Fallback client-side no approveLicense() permite que, se a RPC falhar (por qualquer motivo), a aprovação seja tentada diretamente via query na tabela licenses.

Explorabilidade: Atualmente bloqueado pelo RLS. Risco latente que se materializa se o RLS for alterado ou se o padrão for reutilizado.

```
// Fallback direto via cliente caso a RPC ainda não exista
const { error } = await supabase
  .from('licenses')
  .update({
```

ALTA

F06 — services/adminLicenseService.ts:113-138

Mesmo padrão de fallback client-side no deleteLicense(). Se a RPC delete_license_rpc falhar, a exclusão é tentada diretamente.
Explorabilidade: Idem ao F05. RLS protege, mas o fallback é anti-padrão.

```
// deleteLicense – fallback direto
const { error } = await supabase
  .from('licenses')
  .delete()
```

3. IDOR

MÉDIA

F07 — services/scheduleService.ts:180-190

Soft-delete de schedule por ID sem filtro explícito de user_id no código do serviço.
Explorabilidade: Protegido pelo RLS. Se o RLS for desabilitado ou alterado, se torna IDOR.

```
async deleteSchedule(id: string): Promise<void> {
  const { error } = await supabase
    .from('schedules')
    .update({ deleted_at: new Date().toISOString() })
}
```

MÉDIA

F08 — services/scheduleService.ts:221-231

Restauração de schedule por ID sem filtro explícito de user_id.
Explorabilidade: Idem ao F07.

```
async restoreSchedule(id: string): Promise<void> {
  const { error } = await supabase
    .from('schedules')
    .update({ deleted_at: null })
}
```

MÉDIA

F09 — services/scheduleService.ts:233-243

Exclusão permanente de schedule por ID sem filtro explícito de user_id.
Explorabilidade: Protegido pelo RLS.

```
async permanentlyDeleteSchedule(id: string): Promise<void> {
  const { error } = await supabase
    .from('schedules')
    .delete()
}
```

MÉDIA

F10 — services/notificationService.ts:45-56

Marcar notificação como lida por ID sem filtro de user_id.
Explorabilidade: Protegido pelo RLS. Impacto mínimo mesmo sem RLS.

```
async markAsRead(notificationId: string): Promise<boolean> {
  const { error } = await supabase
    .from('notifications')
    .update({ is_read: true })
}
```

4. Chaves Expostas

CRÍTICA

F11 — .env.local:1

Chave de API do Google Gemini armazenada em .env.local. Embora o .env.local não esteja rastreado pelo git (*.local no .gitignore), a chave foi encontrada no histórico git (commits ff8a553 e c34535d).
Explorabilidade: Qualquer pessoa com acesso ao repositório git pode extrair a chave do histórico e usá-la para fazer chamadas à API do Google, gerando custos ao proprietário.

```
GEMINI_API_KEY=AIzaSy[REDACTED]
```

INFORMATIVA

F12 — services/supabaseClient.ts:3-4

A Supabase Anon Key e URL são expostas no bundle do frontend (prefixo VITE_). Isso é o design intencional do Supabase (anon key é pública por definição), mas a segurança DEPENDE inteiramente do RLS estar corretamente configurado.

Explorabilidade: Esperado e por design. Não é uma vulnerabilidade se o RLS estiver bem configurado.

```
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;
```

ALTA

F13 — schema.sql:9

E-mails de administradores hardcoded na função is_admin() do banco de dados e no frontend (types.ts:14).

Explorabilidade: Facilita ataques de engenharia social direcionados aos e-mails administrativos. Gestão inflexível de privilégios.

```
RETURN (auth.jwt() ->> 'email') IN ('horium.app@gmail.com', 'prof.jackson@gmail.com');
```

Recomendações Priorizadas

[P1] Revogar a chave Gemini API (AlzaSy...) exposta no histórico git e gerar uma nova. Considerar reescrever o histórico git ou tornar o repositório privado.

[P1] Remover TODOS os fallbacks client-side das operações admin (approveLicense, deleteLicense). Se a RPC falhar, deve falhar — não tentar bypass.

[P2] Mover e-mails de admin para variáveis de ambiente ou tabela no banco com RLS, removendo do código-fonte.

[P2] Substituir getPublicUrl() por createSignedUrl() nos buckets privados (receipts e tickets-attachments).

[P2] Adicionar filtro explícito de user_id nas queries do frontend (deleteSchedule, restoreSchedule, permanentlyDeleteSchedule, markAsRead) como defesa em profundidade, mesmo com RLS ativo.

[P2] Usar a RPC get_pending_licenses_count_rpc no getPendingLicensesCount() em vez da query direta.

[P3] Adicionar validação de startup que rejeite valores padrão inseguros para variáveis de ambiente (ex: lançar erro se VITE_SUPABASE_URL estiver vazio).

[P3] Implementar rate limiting nas RPCs administrativas para evitar abuso.

Issues para o GitHub

Abaixo estão as issues completas em formato Markdown, prontas para copiar e colar no GitHub.

--- ISSUE 1 ---

Título: [Segurança] Chave Gemini API exposta no histórico git

Labels: security, critica

Descrição:

A chave de API do Google Gemini ("AlzaSy[REDACTED]") está presente no histórico git do repositório (commits ff8a553 e c34535d). Mesmo não estando no branch atual (.env.local está no .gitignore), qualquer pessoa com acesso ao repositório pode extraí-la.

Evidência:

`env.local:1` - `GEMINI\_API\_KEY=AIzaSy[REDACTED]`
Histórico git: `git log --all -S "AIzaSy" --oneline` retorna commits ff8a553 e c34535d.

Impacto:

Uso não autorizado da API Gemini, gerando custos financeiros ao proprietário da chave.

Sugestão de Correção:

1. Revogar a chave imediatamente no Google Cloud Console
2. Gerar uma nova chave
3. Considerar `git filter-branch` ou `BFG Repo-Cleaner` para limpar o histórico
4. Verificar se o repositório é público; se sim, tratar como comprometida

Critérios de Aceite:

- [] Chave antiga revogada no Google Cloud Console
- [] Nova chave gerada e configurada apenas em .env.local
- [] Histórico git limpo OU repositório marcado como privado
- [] Nenhuma chave sensível aparece em `git log --all -S "AlzaSy"`

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Remover fallbacks client-side das operações admin

Labels: security, alta

Descrição:

Os métodos `approveLicense()` e `deleteLicense()` em `adminLicenseService.ts` contêm fallbacks que tentam operações diretamente na tabela `licenses` quando as RPCs seguras falham. Embora o RLS atualmente bloqueie não-admins, o padrão é inseguro por design e cria dependência frágil.

Evidência:

`services/adminLicenseService.ts:56-101` (approveLicense fallback)
`services/adminLicenseService.ts:124-134` (deleteLicense fallback)

Impacto:

Se o RLS for relaxado no futuro, qualquer usuário autenticado poderia aprovar ou excluir licenças. O fallback também causa inconsistência (não cria notificação nem atualiza schedules).

Sugestão de Correção:

1. Remover os blocos de fallback client-side
2. Se a RPC falhar, propagar o erro ao usuário
3. Garantir que as RPCs estão deployed no Supabase

Critérios de Aceite:

- [] Nenhum fallback client-side para operações de admin em `adminLicenseService.ts`
- [] Erro da RPC é exibido ao admin quando falha
- [] RPCs `approve\_license\_rpc` e `delete\_license\_rpc` verificadas como deployed

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] E-mails de admin hardcoded no código-fonte e no banco

Labels: security, alta

Descrição:

Os e-mails dos administradores estão hardcoded em `schema.sql:9` (função `is\_admin()`) e em `types.ts:14` (`ADMIN\_EMAILS`). Isso expõe a identidade dos admins e dificulta a gestão de privilégios.

Evidência:

```
`schema.sql:9` - `RETURN (auth.jwt() ->> 'email') IN ('horium.app@gmail.com', 'prof.jackison@gmail.com');`
`types.ts:14` - `export const ADMIN_EMAILS = ['horium.app@gmail.com', 'prof.jackison@gmail.com'];`
```

Impacto:

Exposição de identidade administrativa. Gestão inflexível — requer deploy para adicionar/remover admin.

Sugestão de Correção:

1. Criar tabela `admin\_users` com RLS
2. Alterar `is\_admin()` para consultar a tabela
3. No frontend, consultar uma RPC que retorne se o usuário é admin

Critérios de Aceite:

- [] Nenhum e-mail hardcoded no código-fonte
- [] Função `is\_admin()` consulta tabela dinâmica
- [] Frontend usa RPC para verificar papel

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Uso de getPublicUrl() em buckets privados

Labels: security, media

Descrição:

Os métodos de upload de comprovantes (`PlansPage.tsx:146-149`) e tickets (`ticketService.ts:36-40`) usam `getPublicUrl()` para gerar URLs de arquivos em buckets privados. As URLs geradas não funcionam para buckets privados.

Evidência:

```
`services/ticketService.ts:36-40`
`pages/PlansPage.tsx:146-149`
`components/licenses/LicensePurchase.tsx:103`
```

Impacto:

Comprovantes e anexos de tickets não são visualizáveis. Funcionalidade quebrada.

Sugestão de Correção:

1. Substituir `getPublicUrl()` por `createSignedUrl()` com expiração
2. Ou salvar apenas o path relativo e gerar signed URL sob demanda na visualização

Critérios de Aceite:

- [] Nenhum uso de `getPublicUrl()` em buckets privados
- [] Comprovantes e anexos são visualizáveis pelos destinatários corretos
- [] URLs têm expiração definida

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] Defesa em profundidade: adicionar filtro user_id nas queries do frontend

Labels: security, media

Descrição:

Várias operações em `scheduleService.ts` e `notificationService.ts` dependem exclusivamente do RLS para isolamento, sem filtrar por `user\_id` no código. Embora o RLS proteja, a defesa em profundidade recomenda filtro duplo.

Evidência:

```
`scheduleService.ts:180-190` (deleteSchedule)
`scheduleService.ts:221-231` (restoreSchedule)
`scheduleService.ts:233-243` (permanentlyDeleteSchedule)
`notificationService.ts:45-56` (markAsRead)
```

Impacto:

Se o RLS for desabilitado ou alterado incorretamente, as operações se tornam IDOR.

Sugestão de Correção:

Adicionar `.eq('user\_id', user.id)` em todas as queries que operam por ID em tabelas com dados multi-tenant.

Critérios de Aceite:

- [] Todas as queries de mutação filtram por user_id além do ID do recurso
- [] Testes confirmam que queries retornam erro se user_id não corresponde

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] Usar RPC segura para contagem de licenças pendentes

Labels: security, alta

Descrição:

O método `getPendingLicensesCount()` em `adminLicenseService.ts:144-158` faz query direta na tabela `licenses` em vez de usar a RPC `get_pending_licenses_count_rpc` que valida admin.

Evidência:

`services/adminLicenseService.ts:144-158`

Impacto:

Bypass da RPC segura. Resultado incorreto para admins (sub-contagem por causa do RLS).

Sugestão de Correção:

Substituir a query direta pela chamada `supabase.rpc('get_pending_licenses_count_rpc')`.

Critérios de Aceite:

- [] `getPendingLicensesCount()` usa a RPC `get_pending_licenses_count_rpc`
- [] Contagem retorna o total global para admins

--- FIM ISSUE 6 ---

